niftyMomnt

Software Requirements Specification

Modular Clean Architecture · NiftyCore SDK · Multi-Variant iOS Platform

Version	1.0 — Merged & Definitive (reconciles PRD v1.5 + UI/UX Spec v1.6 + Architecture ADR v1.0)
Date	March 2026
Platform	iOS 26.0 · iPhone 15 minimum · Swift 5.9+ · Xcode 16+
Document Status	Final Pre-Development
Companion Docs	PRD v1.5 · UI/UX Spec v1.6 · Architecture ADR v1.0
Author	Han Wook Cho · han@niftymomnt.com
Classification	Confidential

> **Merge note: This document supersedes both the prior SRS v1.0 (architecture-only, PRD v1.3 baseline) and SRS v1.1 (Fix + Sound Stamp spec, PRD v1.5). It is the single source of truth for VCS+Claude coding sessions.**

1. Introduction & Architectural Philosophy

1.1 Purpose

This Software Requirements Specification defines the technical architecture, module boundaries, interface contracts, data models, platform requirements, and feature specifications for niftyMomnt v1.0 and its companion apps. It is the primary reference for iOS engineers and the authoritative source for all implementation decisions. It builds on PRD v1.5, UI/UX Spec v1.6, MRD v1.5, and Architecture ADR v1.0.

Sections 1–3 cover architecture, module structure, and the variant capability system. Sections 4–5 cover domain models and protocol contracts. Sections 6–7 cover engine and data layer specifications. Sections 8–9 cover DI wiring and concurrency. Section 10 covers security and privacy. Section 11 covers non-functional requirements. Sections 12–13 cover the two v1.5-new MVP features — Still Photo Fix and Sound Stamp — in full detail. Section 14 covers resolved open issues.

1.2 Core Architectural Principle — Clean Architecture

> **The UI layer must be completely substitutable without modifying any business logic, data access, or AI processing code. Adding a new app variant is: new Xcode target + new UI layer + new AppConfig. Zero changes to NiftyCore.**

niftyMomnt is built on Clean Architecture adapted for iOS. All business logic lives in a local Swift Package — NiftyCore — that has zero knowledge of UIKit, SwiftUI, or any specific app target. The presentation layer is the only layer that knows about the UI framework. Every cross-layer dependency is expressed as a Swift protocol defined in the domain layer and implemented in the data layer.

1.3 The Dependency Rule

Layer	Contents	May import	Must never import
1 — App Target	Xcode target, entry point, DI wiring, AppConfig	Presentation, Domain, Data, NiftyCore	Nothing — it owns everything
2 — Presentation	SwiftUI Views, ViewModels, Coordinators	Domain (protocols only), NiftyCore models	Data layer implementations, iOS frameworks directly
3 — Domain (NiftyCore)	Use cases, business logic, protocol definitions, domain models	Swift stdlib only	UIKit, SwiftUI, Foundation, AVFoundation, CoreML — zero platform imports
4 — Data (NiftyData)	Repository implementations, network clients, persistence	Domain, Foundation, platform frameworks	UIKit, SwiftUI, Presentation layer
5 — Platform	AVFoundation, CoreML, CoreData, WeatherKit, WidgetKit, etc.	Apple SDKs	Everything above — platform frameworks wrapped in Data layer adapters only

The dependency rule is enforced by CI, not by convention. A SwiftLint custom rule scans every file in NiftyCore/Sources/ and fails the build if it finds any import of UIKit, SwiftUI, AVFoundation, CoreML, or any Apple platform framework.

1.4 Multi-Variant Strategy

Variant	Asset types	AI modes	UI style	Primary use case
niftyMomnt (full)	All 5	Mode 0+1+2	Full MZ aesthetic UI	Primary consumer app
niftyMomnt Lite	Still + Clip only	Mode 0 only	Minimal, fast UI	Onboarding funnel / reduced friction entry
Future variant	Configurable	Configurable	Custom theme	Defined via AppConfig at target creation

Adding a new variant requires: new Xcode target + new UI layer + new AppConfig extension. The CaptureEngine, IndexingEngine, StoryEngine, NudgeEngine, VaultManager, GraphManager, and all NiftyCore code are touched zero times.

2. Package & Module Structure

2.1 Repository Layout

The monorepo is structured so that NiftyCore is a local Swift Package dependency shared by all app targets. No app target contains any business logic.

```
niftyMomnt/
├── NiftyCore/                      ← Local Swift Package (all business logic)
│   ├── Package.swift
│   ├── Sources/
│   │   ├── Domain/                 ← Pure Swift — zero platform imports
│   │   │   ├── Models/             ← Moment, Asset, VibeTag, DerivativeAsset...
│   │   │   ├── UseCases/           ← CaptureUseCase, FixUseCase, IndexUseCase...
│   │   │   ├── Protocols/          ← CaptureEngineProtocol, VaultProtocol...
│   │   │   └── AppConfig.swift     ← Capability set, feature flags
│   │   ├── Engines/                ← Business logic implementations
│   │   │   ├── CaptureEngine.swift
│   │   │   ├── IndexingEngine.swift
│   │   │   ├── StoryEngine.swift
│   │   │   ├── NudgeEngine.swift
│   │   │   └── VoiceProseEngine.swift
│   │   └── Managers/               ← Cross-cutting services
│   │       ├── VaultManager.swift
│   │       ├── GraphManager.swift
│   │       └── LabClient.swift
│   └── Tests/
├── NiftyData/                      ← Local Swift Package (data layer)
│   ├── Sources/
│   │   ├── Repositories/           ← Implements NiftyCore protocols
│   │   ├── Persistence/            ← CoreData stack, SQLCipher graph
│   │   ├── Network/                ← LabNetworkClient, EnhancedAIClient
│   │   └── Platform/               ← AVFoundation, CoreML, WeatherKit adapters
│   └── Tests/
├── Apps/
│   ├── niftyMomnt/                 ← Full app target
│   │   ├── niftyMomntApp.swift     ← Entry point, DI container wiring
│   │   ├── AppConfig+Full.swift    ← Full capability set
│   │   └── UI/                     ← All SwiftUI views for this variant
│   ├── niftyMomntLite/
│   │   ├── niftyMomntLiteApp.swift
│   │   ├── AppConfig+Lite.swift
│   │   └── UI/
│   └── Widgets/                    ← WidgetKit extension (shared)
└── CompanionApps/
    ├── TripPlanner/
    ├── AnnualReview/
    ├── TravelPortfolio/
    └── SharedJourney/
```

2.2 NiftyCore Module Breakdown

Module	Responsibility	Key types	Platform imports
Domain/Models	All value types shared across the system	Moment, Asset, AssetType, DerivativeAsset, VibeTag,	None — Swift stdlib only

		AcousticTag, MoodPoint, PlaceRecord	
Domain/Protocols	All cross-layer contracts as Swift protocols	CaptureEngineProtocol, VaultProtocol, GraphProtocol, FixRepositoryProtocol, SoundStampPipelineProtocol, LabClientProtocol, IndexingProtocol	None — Swift stdlib only
Domain/UseCases	Orchestrates protocols to implement user-facing operations	CaptureMomentUseCase, FixAssetUseCase, AssembleReelUseCase, GenerateNudgeUseCase, ShareMomentUseCase	None — pure Swift
Domain/AppConfig	Capability set and feature flags per variant	AppConfig, FeatureSet (incl. .photoFix, .soundStamp), AssetTypeSet, AIModeSet	None
Engines/Capture	Manages capture session lifecycle, mode switching, Sound Stamp orchestration, live telemetry	CaptureEngine, CaptureMode, CaptureState, TelemetryPublisher, SoundStampOrchestrator	None — delegates to protocols
Engines/Indexing	On-device classification, clustering, ambient harvesting	IndexingEngine, MomentCluster, VibeClassifier, ChromaticProfiler	None
Engines/Story	Reel assembly, journal organization, caption generation	StoryEngine, ReelAssembler, AssetScorer, CaptionGenerator	None
Engines/Nudge	Trigger evaluation, question generation, response handling	NudgeEngine, NudgeTrigger, NudgeCard, QuestionGenerator	None
Managers/Vault	Encrypted asset storage lifecycle; Fix derivative management	VaultManager, VaultItem, VaultQuery, DerivativeRecord	None — delegates to VaultProtocol
Managers/Graph	Intelligence graph read/write, App Group export	GraphManager, GraphQuery, GraphExporter	None
Managers/Lab	Cloud VLM session management, consent, purge verification	LabClient, LabSession, LabConsent, LabResult	None — delegates to LabClientProtocol

3. AppConfig — Variant Capability System

3.1 Overview

AppConfig is a value type injected at app startup. Every engine checks AppConfig before activating any capability. No capability is hard-coded. The .photoFix and .soundStamp flags are v1.5 additions — both are included in AppConfig.full and absent from AppConfig.lite.

3.2 AppConfig Definition

```
// NiftyCore/Sources/Domain/AppConfig.swift

public struct AppConfig {
    public let appVariant:  AppVariant
    public let assetTypes:  AssetTypeSet
    public let aiModes:     AIModeSet
    public let features:    FeatureSet
    public let sharing:     SharingConfig
    public let storage:     StorageConfig
}

public struct AssetTypeSet: OptionSet {
    public static let still      = AssetTypeSet(rawValue: 1 << 0)
    public static let live       = AssetTypeSet(rawValue: 1 << 1)
    public static let clip       = AssetTypeSet(rawValue: 1 << 2)
    public static let echo       = AssetTypeSet(rawValue: 1 << 3)
    public static let atmosphere = AssetTypeSet(rawValue: 1 << 4)
    public static let all: AssetTypeSet = [.still,.live,.clip,.echo,.atmosphere]
    public static let basic: AssetTypeSet = [.still,.clip]
}

public struct AIModeSet: OptionSet {
    public static let onDevice   = AIModeSet(rawValue: 1 << 0)  // Mode 0 — always
on
    public static let enhancedAI = AIModeSet(rawValue: 1 << 1)  // Mode 1 — text
only
    public static let lab        = AIModeSet(rawValue: 1 << 2)  // Mode 2 —
encrypted visual
    public static let full: AIModeSet = [.onDevice,.enhancedAI,.lab]
}

public struct FeatureSet: OptionSet {
    public static let rollMode       = FeatureSet(rawValue: 1 << 0)
    public static let nudgeEngine    = FeatureSet(rawValue: 1 << 1)
    public static let moodMap        = FeatureSet(rawValue: 1 << 2)
    public static let liveActivity   = FeatureSet(rawValue: 1 << 3)
    public static let journalSuggest = FeatureSet(rawValue: 1 << 4)
    public static let trustedSharing = FeatureSet(rawValue: 1 << 5)
    public static let widgetKit      = FeatureSet(rawValue: 1 << 6)
    public static let photoFix       = FeatureSet(rawValue: 1 << 7)  // NEW v1.5
    public static let soundStamp     = FeatureSet(rawValue: 1 << 8)  // NEW v1.5
    public static let all: FeatureSet = [.rollMode,.nudgeEngine,.moodMap,
                                         .liveActivity,.journalSuggest,
                                         .trustedSharing,.widgetKit,
                                         .photoFix,.soundStamp]
}
```

3.3 Variant Configurations

```
// Apps/niftyMomnt/AppConfig+Full.swift
extension AppConfig {
    static let full = AppConfig(
        appVariant: .full,
        assetTypes: .all,
        aiModes:    .full,
        features:   .all,            // includes .photoFix + .soundStamp
        sharing:    SharingConfig(maxCircleSize: 10, labEnabled: true),
        storage:    StorageConfig(smartArchiveEnabled: true, iCloudSyncEnabled:
true)
    )
}

// Apps/niftyMomntLite/AppConfig+Lite.swift
extension AppConfig {
    static let lite = AppConfig(
        appVariant: .lite,
        assetTypes: .basic,           // Still + Clip only
        aiModes:    [.onDevice],
        features:   [.rollMode, .widgetKit],   // no photoFix, no soundStamp
        sharing:    SharingConfig(maxCircleSize: 5, labEnabled: false),
        storage:    StorageConfig(smartArchiveEnabled: false, iCloudSyncEnabled:
false)
    )
}
```

4. Domain Models

4.1 Core Types

All domain models are pure Swift value types (structs or enums). They carry no persistence annotations, no Codable conformances in the domain layer, and no UI-related properties.

Asset — the atomic capture unit

```
public struct Asset: Identifiable, Equatable {
    public let id:             UUID
    public let type:           AssetType
    public let capturedAt:     Date
    public let location:       GPSCoordinate?
    public var vibeTags:       [VibeTag]
    public var acousticTags:   [AcousticTag]
    public var palette:        ChromaticPalette?
    public var ambient:        AmbientMetadata
    public var transcript:     String?          // Echo only
    public var score:          AssetScore?      // Set by StoryEngine
    public var duration:       TimeInterval?    // Clip / Echo / Atmosphere
    public var derivative:     DerivativeAsset? // Set by Fix — nil if no fix
applied
}
```

DerivativeAsset — Fix output (NEW v1.5)

```
// NiftyCore/Sources/Domain/Models/DerivativeAsset.swift
public struct DerivativeAsset: Equatable {
    public let id:              UUID
    public let sourceAssetID:   UUID
    public let createdAt:       Date
    public var cropRect:        NormalizedRect?    // nil = no crop applied
    public var rotationDegrees: Int                // 0, 90, 180, 270 (clockwise)
    public var flipHorizontal:  Bool
    public var flipVertical:    Bool
}

// Normalized 0.0-1.0 coordinate space
public struct NormalizedRect: Equatable {
    public let x: Double; public let y: Double
    public let width: Double; public let height: Double
}
```

Moment — a cluster of related assets

```
public struct Moment: Identifiable, Equatable {
    public let id:            UUID
    public var label:         String       // e.g. "Rainy Walk · Siena · Tuesday"
    public var assets:        [Asset]
    public var centroid:      GPSCoordinate
    public var startTime:     Date
    public var endTime:       Date
    public var dominantVibes: [VibeTag]
    public var moodPoint:     MoodPoint?
```

```
    public var isStarred:     Bool
    public var heroAssetID:   UUID?
}
```

AmbientMetadata, AcousticTag, supporting types

```
public struct AmbientMetadata: Equatable {
    public var weather:           WeatherCondition?
    public var temperatureC:      Double?
    public var elevationM:        Double?
    public var sunPosition:       SunPosition?
    public var nowPlayingTrack:   String?
    public var nowPlayingArtist:  String?
}

public struct AcousticTag: Equatable {
    public let tag:        AcousticTagType
    public let source:     AcousticSource   // .soundStamp | .echo | .clip |
.atmosphere
    public let confidence: Float            // 0.0-1.0
}

public enum AcousticTagType: String, CaseIterable {
    case windy, crowded, music, quiet, serene, rain, ocean, water
}

public enum AcousticSource: String {
    case soundStamp, echo, clip, atmosphere
}

public struct AssetScore: Equatable {
    public let motionInterest:   Double  // weight 0.30
    public let vibeCoherence:    Double  // weight 0.30
    public let chromaticHarmony: Double  // weight 0.20
    public let uniqueness:       Double  // weight 0.20
    public var composite: Double { (motionInterest + vibeCoherence
                                   + chromaticHarmony + uniqueness) / 4 }
}
```

4.2 Intelligence Graph Models

```
public struct PlaceRecord: Identifiable, Equatable {
    public let id:             UUID
    public let placeName:      String
    public let coordinate:     GPSCoordinate
    public var visitCount:     Int
    public var totalDwellMins: Int
    public var firstVisit:     Date
    public var lastVisit:      Date
    public var dominantVibes:  [VibeTag]
}

public struct MoodPoint: Equatable {
    public let momentID:     UUID
```

```
    public let coordinate:   GPSCoordinate
    public let dominantMood: MoodTag
    public let palette:      [EmotionColor]
}
```

5. Protocol Contracts — Cross-Layer Boundaries

5.1 Overview

Every dependency that crosses a layer boundary is expressed as a Swift protocol defined in the domain layer. The data layer provides concrete implementations. Presentation and engines depend only on protocols — never on implementations. This is what makes each layer independently testable and swappable.

> **Protocol-first rule: any new cross-layer capability must be introduced as a protocol in Domain/Protocols/ before any implementation is written. Protocol reviewed before implementation sprint begins.**

5.2 CaptureEngineProtocol

```
public protocol CaptureEngineProtocol: AnyObject {
    var captureState: AnyPublisher<CaptureState, Never> { get }
    var telemetry:    AnyPublisher<CaptureTelemetry, Never> { get }
    func startSession(mode: CaptureMode, config: AppConfig) async throws
    func stopSession() async
    func captureAsset() async throws -> Asset
    func switchMode(to mode: CaptureMode) async throws
    func applyPreset(_ preset: VibePreset) async
}

public struct CaptureTelemetry {
    public let mode:       CaptureMode
    public let elapsed:    TimeInterval
    public let ceiling:    TimeInterval
    public let isWarning:  Bool          // elapsed >= ceiling - 5.0
    public let audioLevel: Float?        // Echo only: 0.0-1.0
}
```

5.3 VaultProtocol

```
public protocol VaultProtocol: AnyObject {
    func save(_ asset: Asset, data: Data) async throws
    func saveDerivative(_ derivative: DerivativeAsset,
                        data: Data,
                        sourceAssetID: UUID) async throws
    func load(_ assetID: UUID) async throws -> (Asset, Data)
    func loadPrimary(_ assetID: UUID) async throws -> (Asset, Data)
                        // returns derivative if exists, else original
    func deleteDerivative(for assetID: UUID) async throws
    func delete(_ assetID: UUID) async throws
    func query(_ query: VaultQuery) async throws -> [Asset]
    func exportToPhotoLibrary(_ assetID: UUID) async throws
    var storageUsedBytes: AnyPublisher<Int64, Never> { get }
}
```

5.4 FixRepositoryProtocol (NEW v1.5)

```
// NiftyCore/Sources/Domain/Protocols/FixRepositoryProtocol.swift
public protocol FixRepositoryProtocol: AnyObject {
    /// Apply geometry corrections and write derivative to vault.
    /// Returns the new DerivativeAsset.
    func applyFix(to assetID: UUID,
                  cropRect: NormalizedRect?,
                  rotationDegrees: Int,
                  flipH: Bool,
                  flipV: Bool) async throws -> DerivativeAsset

    /// Remove derivative and restore original as primary.
    func revertFix(for assetID: UUID) async throws

    /// Returns the current derivative for an asset, if one exists.
    func derivative(for assetID: UUID) async throws -> DerivativeAsset?
}
```

5.5 SoundStampPipelineProtocol (NEW v1.5)

```
// NiftyCore/Sources/Domain/Protocols/SoundStampPipelineProtocol.swift
public protocol SoundStampPipelineProtocol: AnyObject {
    /// Start the always-on pre-roll buffer.
    /// Called on entry to Still mode when soundStamp feature is enabled.
    func activatePreRoll() async throws

    /// Stop pre-roll buffer and release microphone session.
    /// Called on mode switch, app background, or feature toggle-off.
    func deactivatePreRoll() async

    /// Capture the 1.0s post-shutter buffer, combine with 0.5s pre-roll,
    /// run acoustic classification, write tags to graph, release buffer.
    /// Returns acoustic tags written. Does not block caller — fire-and-forget
safe.
    func analyzeAndTag(assetID: UUID) async throws -> [AcousticTag]

    /// Current activation state (for UI indicator binding).
    var isActive: AnyPublisher<Bool, Never> { get }
}
```

5.6 GraphProtocol

```
public protocol GraphProtocol: AnyObject {
    func saveMoment(_ moment: Moment) async throws
    func updateVibeTag(_ tag: VibeTag, for assetID: UUID) async throws
    func updateAcousticTag(_ tag: AcousticTag, for assetID: UUID) async throws
    func saveNudgeResponse(_ response: NudgeResponse) async throws
    func saveMoodPoint(_ point: MoodPoint) async throws
    func updatePlaceRecord(_ record: PlaceRecord) async throws
    func saveDerivativeRecord(_ derivative: DerivativeAsset) async throws
    func deleteDerivativeRecord(for assetID: UUID) async throws
    func fetchMoments(query: GraphQuery) async throws -> [Moment]
```

```
    func fetchPlaceHistory(limit: Int) async throws -> [PlaceRecord]
    func fetchMoodMap(range: DateInterval) async throws -> [MoodPoint]
    func exportForCompanion() async throws -> GraphExport
}
```

5.7 IndexingProtocol

```
public protocol IndexingProtocol: AnyObject {
    func classifyImage(_ assetID: UUID, imageData: Data) async throws -> [VibeTag]
    func analyzeAudio(_ assetID: UUID, audioData: Data) async throws ->
[AcousticTag]
    func analyzePCMBuffer(_ assetID: UUID,
                          buffer: UnsafeBufferPointer<Float>,
                          sampleRate: Double) async throws -> [AcousticTag]
                          // ↑ used by SoundStamp — in-memory PCM, never a file
    func extractPalette(_ assetID: UUID, imageData: Data) async throws ->
ChromaticPalette
    func harvestAmbientMetadata(at location: GPSCoordinate?,
                                at time: Date) async throws -> AmbientMetadata
    func clusterMoments(assets: [Asset]) async throws -> [Moment]
}
```

5.8 LabClientProtocol & NudgeEngineProtocol

These protocols are unchanged from the PRD v1.4 baseline. Full definitions in NiftyCore/Sources/Domain/Protocols/. Key signatures:

```
public protocol LabClientProtocol: AnyObject {
    func generateCaption(for moment: Moment, tone: CaptionTone) async throws ->
[CaptionCandidate]
    func transformProse(_ transcript: String, styles: [ProseStyle]) async throws ->
[ProseVariant]
    func requestLabSession(assets: [UUID], consent: LabConsent) async throws ->
LabSession
    func processLabSession(_ session: LabSession) async throws -> LabResult
    func verifyPurge(sessionID: UUID) async throws -> PurgeConfirmation
}

public protocol NudgeEngineProtocol: AnyObject {
    var pendingNudge: AnyPublisher<NudgeCard?, Never> { get }
    func evaluateTriggers(for moment: Moment) async
    func submitResponse(_ response: NudgeResponse) async throws
    func dismiss(nudgeID: UUID)
    func snooze(nudgeID: UUID, until: Date)
}
```

6. Engine Specifications

6.1 CaptureEngine

- **Responsibilities:** Owns the capture session lifecycle — start, stop, mode switch. Publishes real-time CaptureState and CaptureTelemetry. Implements mode navigation logic (swipe model, ghost label trigger, available modes filtered by AppConfig). Orchestrates Sound Stamp pipeline activation/deactivation on Still mode entry/exit. Never imports AVFoundation.

State	Entry condition	Exit condition	Published telemetry
idle	App launch / session end	startSession() called	None
ready	Session started successfully	captureAsset() or switchMode()	Current mode, available modes
capturing	captureAsset() called	Asset complete or error	CaptureTelemetry: elapsed, ceiling, isWarning, audioLevel
processing	Raw capture complete	Asset saved to vault	None — async background
error	Any recoverable failure	Error cleared / session restarted	CaptureError type

Sound Stamp integration in CaptureEngine:

- On switchMode(to: .still): if config.features.contains(.soundStamp) && soundStampEnabled, call soundStampPipeline.activatePreRoll().
- On switchMode(away from .still) or stopSession() or app background: call soundStampPipeline.deactivatePreRoll().
- On captureAsset() in Still mode with Sound Stamp active: after asset is written to vault, fire soundStampPipeline.analyzeAndTag(assetID:) as a non-blocking async task.

6.2 IndexingEngine

Runs entirely on-device via BGProcessingTask (requires charging + idle). Processes unindexed assets from vault in FIFO order. Calls IndexingProtocol methods for classification, audio analysis, chromatic profiling, and ambient harvesting. Never directly writes to persistence — all results via GraphProtocol.

#	Step	Input	Output to graph
1	Image classification (Still, Live, Clip, Atmosphere)	Image data from vault	Place tags, vibe tags, aesthetic tags
2	Acoustic analysis (Echo, Clip audio, Atmosphere audio)	Audio data from vault	Acoustic tags (source: .echo/.clip/.atmosphere)
3	Sound Stamp acoustic analysis (Still — if Sound Stamp enabled)	In-memory PCM buffer via SoundStampPipelineProtocol — never from vault	Acoustic tags (source: .soundStamp) — written within 300ms of buffer end

4	Chromatic profiling (all visual types)	Image data from vault	ChromaticPalette — up to 5 HSL colors
5	Ambient metadata harvest	GPS coordinate, capture timestamp	AmbientMetadata (weather, elevation, sun position, Now Playing)
6	Speech transcription (Echo assets)	Audio data from vault	transcript string on Asset
7	Moment clustering (batch — after 3+ assets indexed)	All indexed assets in rolling 90-min / 200m window	Moment records with label, centroid, dominantVibes

6.3 StoryEngine — AssetScorer Weights

Factor	Weight	Derivation
Motion interest	0.30	Clips with significant subject motion: 0.8–1.0. Stills and Lives: 0.3–0.6 based on composition.
Vibe coherence	0.30	Overlap between asset vibe tags and Moment dominant vibes / total Moment vibe count.
Chromatic harmony	0.20	HSL distance between asset palette and Moment cluster palette centroid.
Uniqueness	0.20	1.0 minus average cosine similarity to all other assets in cluster. Penalises near-duplicates.

7. Data Layer Specifications

7.1 VaultRepository — Encrypted Asset Storage

Attribute	Specification
Encryption	AES-256-GCM. Each asset encrypted with a unique per-asset data encryption key (DEK). DEK wrapped with master key stored in iOS Keychain with kSecAttrAccessibleWhenUnlockedThisDeviceOnly. Never transmitted.
Fix derivatives	Derivative encrypted with the same DEK as its source original. Both files co-located in vault. Revert deletes derivative file and DEK reference. derivatives never included in iCloud offload / Smart Archive pipeline (v1.0 decision — OI-002 resolved).
Asset file naming	UUID-based: {uuid}.still.enc, {uuid}.clip.enc, {uuid}.still.fix.enc (derivatives), etc.
Storage backend	FileManager-managed app sandbox directory. CoreData with NSPersistentCloudKitContainer for metadata (optional iCloud sync — metadata only, not encrypted binary files).
Query interface	VaultQuery value type with filter, sort, pagination. Implemented as NSFetchRequest — never exposed to domain layer.
Smart archive	BGAppRefreshTask. Compresses Stills older than threshold to JPEG 80%. Fix derivatives excluded from Smart Archive.

7.2 GraphRepository — Intelligence Graph Schema

Attribute	Specification
Database	SQLite via SQLCipher AES-256. Located in iOS App Group shared container (com.niftymomnt.group). Accessible to companion apps with same Team ID.
Write access	NiftyCore exclusive. SQLite WAL mode enabled. Companion apps open read-only connections.
Schema version	Integer schema_version in metadata table. Incremented on any schema change.

Core table schema (DDL):

```
CREATE TABLE moments (id TEXT PRIMARY KEY, label TEXT, centroid_lat REAL,
    centroid_lng REAL, start_time INTEGER, end_time INTEGER,
    hero_asset_id TEXT, is_starred INTEGER DEFAULT 0);

CREATE TABLE vibe_tags (id INTEGER PRIMARY KEY AUTOINCREMENT,
    asset_id TEXT NOT NULL, tag TEXT NOT NULL, source TEXT NOT NULL,
    -- source values: 'sound_stamp' | 'echo' | 'clip' | 'atmosphere' |
'image_classifier'
    confidence REAL NOT NULL, created_at INTEGER NOT NULL);
```

```
CREATE INDEX idx_vibe_tags_asset ON vibe_tags(asset_id);

CREATE TABLE asset_scores (asset_id TEXT PRIMARY KEY,
    fix_applied INTEGER DEFAULT 0,
    is_derivative INTEGER DEFAULT 0,
    source_asset_id TEXT,
    fix_crop_x REAL, fix_crop_y REAL, fix_crop_w REAL, fix_crop_h REAL,
    fix_rotation_degrees INTEGER DEFAULT 0,
    fix_flip_horizontal INTEGER DEFAULT 0,
    fix_flip_vertical INTEGER DEFAULT 0,
    created_from_fix INTEGER,
    composite_score REAL,
    motion_interest REAL, vibe_coherence REAL,
    chromatic_harmony REAL, uniqueness REAL);
CREATE INDEX idx_asset_scores_source ON asset_scores(source_asset_id);

CREATE TABLE ambient_metadata (asset_id TEXT PRIMARY KEY,
    weather TEXT, temperature_c REAL, elevation_m REAL,
    sun_position TEXT, now_playing_track TEXT, now_playing_artist TEXT);

CREATE TABLE nudge_responses (id TEXT PRIMARY KEY, moment_id TEXT,
    question_text TEXT, response_type TEXT, response_value TEXT,
    timestamp INTEGER);

CREATE TABLE place_history (id TEXT PRIMARY KEY, place_name TEXT,
    lat REAL, lng REAL, visit_count INTEGER, total_dwell_mins INTEGER,
    first_visit INTEGER, last_visit INTEGER);

CREATE TABLE metadata (key TEXT PRIMARY KEY, value TEXT);
-- Initial row: INSERT INTO metadata VALUES ('schema_version', '1');
```

7.3 Platform Adapters

Adapter	Implements	Wraps
AVCaptureAdapter	CaptureEngineProtocol	AVFoundation: AVCaptureSession, AVCaptureDevice, AVAssetWriter, AVAudioRecorder. All camera/microphone lifecycle.
SoundStampAdapter	SoundStampPipelineProtocol	AVAudioSession (always-on pre-roll buffer, PCM 44.1kHz). SoundAnalysis.SNClassifySoundRequest for acoustic classification. Never writes audio to disk. Buffer max in-memory lifetime ~500ms.
CoreMLIndexingAdapter	IndexingProtocol (classification, audio, palette)	Vision.VNClassifyImageRequest, SoundAnalysis.SNClassifySoundRequest, CoreImage for palette extraction. All inference on Neural Engine.
SpeechAdapter	IndexingProtocol (transcript)	Speech.SFSpeechRecognizer with on-device recognition. Echo assets only.

WeatherKitAdapter	IndexingProtocol (ambient harvest)	WeatherKit.WeatherService. 30-minute cache. Returns nil on cache miss after 6h.
MapKitGeocoderAdapter	IndexingProtocol (ambient harvest)	CLGeocoder.reverseGeocodeLocation(). Falls back to formatted coordinates on cache miss.
MusicKitAdapter	IndexingProtocol (ambient harvest)	MusicKit.MusicPlayer.Queue.currentEntry. Fallback: SpotifyiOS SDK if authorized.
JournalSuggestionsAdapter	NudgeEngineProtocol (trigger source)	JournalingSuggestions framework (iOS 17.2+, available iOS 26).
LabNetworkAdapter	LabClientProtocol	URLSession TLS 1.3 + certificate pinning. Text-only Enhanced AI + encrypted-asset Lab sessions.
CoreImageFixAdapter	FixRepositoryProtocol	Core Image / Metal for real-time crop/rotate/flip preview and final derivative render. Never imports UIKit.

8. Dependency Injection & Composition Root

8.1 Overview

All dependencies are injected at the composition root — the app entry point. No engine or use case creates its own dependencies. No singletons in NiftyCore. Every component is independently testable: swap in a mock implementation conforming to the protocol and the engine has no knowledge of the change.

8.2 Composition Root Pattern

```
// Apps/niftyMomnt/niftyMomntApp.swift
@main
struct NiftyMomntApp: App {
    private let container: AppContainer
    init() {
        let config = AppConfig.full

        // Platform adapters (NiftyData)
        let captureAdapter    = AVCaptureAdapter(config: config)
        let soundStampAdapter = SoundStampAdapter(config: config)
        let indexingAdapter   = CoreMLIndexingAdapter(config: config)
        let fixAdapter        = CoreImageFixAdapter(config: config)
        let vaultRepo         = VaultRepository(config: config)
        let graphRepo         = GraphRepository(config: config)
        let labClient         = LabNetworkAdapter(config: config)
        let nudgeTrigger      = JournalSuggestionsAdapter(config: config)

        // Core engines (NiftyCore)
        let captureEngine     = CaptureEngine(config: config,
                                              captureAdapter: captureAdapter,
                                              soundStampPipeline:
soundStampAdapter)
        let indexingEngine    = IndexingEngine(config: config,
                                              adapter: indexingAdapter,
                                              graph: graphRepo)
        let storyEngine       = StoryEngine(config: config,
                                            vault: vaultRepo,
                                            graph: graphRepo,
                                            lab: labClient)
        let nudgeEngine       = NudgeEngine(config: config,
                                            graph: graphRepo,
                                            lab: labClient,
                                            triggerSource: nudgeTrigger)
        let vaultManager      = VaultManager(vault: vaultRepo)
        let graphManager      = GraphManager(graph: graphRepo)

        // Use cases
        let captureUseCase    = CaptureMomentUseCase(engine: captureEngine,
                                                     vault: vaultManager,
                                                     indexing: indexingEngine)
        let fixUseCase        = FixAssetUseCase(fixRepo: fixAdapter,
                                                vault: vaultManager,
                                                graph: graphManager)
        let storyUseCase      = AssembleReelUseCase(engine: storyEngine)
        let shareUseCase      = ShareMomentUseCase(vault: vaultManager, config:
config)
```

```
        container = AppContainer(config: config,
                                 captureUseCase: captureUseCase,
                                 fixUseCase: fixUseCase,
                                 storyUseCase: storyUseCase,
                                 shareUseCase: shareUseCase,
                                 nudgeEngine: nudgeEngine,
                                 vaultManager: vaultManager,
                                 graphManager: graphManager)
    }
    var body: some Scene {
        WindowGroup { RootView(container: container) }
    }
}
```

8.3 Testing Strategy

Because every dependency is injected via protocol, unit tests never touch real hardware, file systems, or networks.

Layer	Coverage target	Approach
NiftyCore Domain (models, use cases)	≥ 85% line coverage	Pure Swift unit tests. No mocks needed — value types.
NiftyCore Engines (Capture, Indexing, Story, Nudge)	≥ 80% line coverage	MockCaptureAdapter, MockIndexingAdapter, etc. conforming to protocols. Hardware excluded from coverage requirement.
NiftyData adapters (AVCapture, SoundStamp, CoreMLIndexing, Fix)	≥ 60% line coverage	Integration tests on-device. Some hardware-dependent paths excluded.
Presentation (ViewModels)	≥ 70% line coverage	Mock use cases injected. @MainActor test support.

```
// Tests/NiftyCore/CaptureEngineTests.swift
final class CaptureEngineTests: XCTestCase {
    func test_availableModes_lite_excludesEchoAndAtmosphere() {
        let engine = CaptureEngine(config: .lite,
                                   captureAdapter: MockCaptureAdapter(),
                                   soundStampPipeline: MockSoundStampPipeline())
        let modes = engine.availableModes()
        XCTAssertFalse(modes.contains(.echo))
        XCTAssertFalse(modes.contains(.atmosphere))
        XCTAssertTrue(modes.contains(.still))
    }
    func test_soundStamp_notActivated_whenFeatureFlagOff() async throws {
        let mockPipeline = MockSoundStampPipeline()
        let engine = CaptureEngine(config: .lite,   // .lite has no .soundStamp
flag
                                   captureAdapter: MockCaptureAdapter(),
                                   soundStampPipeline: mockPipeline)
        try await engine.startSession(mode: .still, config: .lite)
        XCTAssertFalse(mockPipeline.preRollActivated)
    }
```

```
}
```

9. Concurrency Model

9.1 Overview

niftyMomnt uses Swift Concurrency (async/await, actors, structured concurrency) exclusively. No manual GCD dispatch queues, no OperationQueue. Combine is used only for reactive UI binding — all async work uses async/await. This applies to the Sound Stamp pipeline and Fix derivative operations.

9.2 Actor Isolation

Component	Isolation	Rationale
CaptureEngine	@MainActor	Capture state must be read from main thread for UI binding. AVCaptureSession callbacks bridged to main via await.
SoundStampAdapter	actor SoundStampAdapter	Serialises pre-roll buffer lifecycle — prevents concurrent activate/deactivate races. Buffer allocated, passed to classifier, and released within a single actor-isolated async call.
IndexingEngine	actor IndexingEngine	Serialises background indexing operations. Prevents concurrent writes to graph from overlapping passes.
VaultManager	actor VaultManager	Serialises all vault reads and writes. Prevents torn writes during concurrent save + smart archive.
GraphManager	actor GraphManager	Serialises all graph mutations. SQLite WAL handles concurrent reads but mutations serialised.
NudgeEngine	@MainActor	NudgeCard publisher subscribed by ViewModel on main thread.
StoryEngine	Nonisolated	Stateless scoring and assembly — pure functional, no shared mutable state.
FixAssetUseCase	Nonisolated	Coordinates between VaultManager and GraphManager actors via await. No shared state itself.

9.3 Background Task Strategy

- IndexingEngine: BGProcessingTask (requires charging + idle). Maximum 60s per pass. Up to 50 assets per pass, then reschedules.
- Smart archive: BGAppRefreshTask. Device charging. Batches of 10. Fix derivatives excluded.
- Graph export: triggered on app backgrounding via scenePhase observer. Maximum 2s. Exports GraphExport JSON to App Group container.

- Sound Stamp analysis: inline async task fired from CaptureEngine post-shutter. Completes within ~500ms. Never a background task — too short.
- Lab session: user-initiated Task with structured cancellation. Cancelled if app backgrounded; user notified on next foreground.

10. Security & Privacy Architecture

10.1 Encryption Stack

Asset / data	Encryption	Key management
Raw assets (vault)	AES-256-GCM. Each asset has a unique per-asset DEK.	DEK wrapped with master key in iOS Keychain (kSecAttrAccessibleWhenUnlockedThisDeviceOnly). Never transmitted.
Fix derivatives	AES-256-GCM. Same DEK as source original.	Same master key. Revert deletes derivative file and removes DEK reference. Never in Smart Archive.
Intelligence graph	SQLCipher AES-256. Database-level encryption.	Database key in iOS Keychain, shared within App Group via kSecAttrAccessGroup. Same-Team-ID apps only.
Lab session (transit)	TLS 1.3 + certificate pinning. Assets additionally AES-256-GCM encrypted on-device before transmission.	Session key derived ephemerally via DH exchange. Never stored.
Enhanced AI (transit)	TLS 1.3 + certificate pinning. Text-only — no asset encryption needed.	Standard TLS certificate pinned to niftyMomnt server CA.
iCloud backup (optional)	NSFileProtectionCompleteUnlessOpen. CloudKit end-to-end encryption for private database.	Apple manages CloudKit keys in user's iCloud keychain. niftyMomnt has no server-side key access.

10.2 Privacy Framework Requirements

Framework	Permission	Info.plist description
Camera	NSCameraUsageDescription	niftyMomnt uses your camera to capture moments.
Microphone	NSMicrophoneUsageDescription	niftyMomnt records ambient sound for Echo and Atmosphere moments, and uses a brief audio buffer at Still capture to tag the ambient vibe of the moment (Sound Stamp). Audio is never stored or shared.
Location (when in use)	NSLocationWhenInUseUsageDescription	niftyMomnt uses your location to tag moments and organise your journal.
Photo Library (add only)	NSPhotoLibraryAddUsageDescription	niftyMomnt saves Live Photos to your photo library when you export them.
Speech Recognition	NSSpeechRecognitionUsageDescription	niftyMomnt transcribes your voice notes on-device to create text.

Apple Music (MusicKit)	NSAppleMusicUsageDescription	niftyMomnt remembers what you were listening to when you capture a moment.
Journaling Suggestions	JSAuthorizationStatus	niftyMomnt uses iOS Journaling Suggestions to prompt reflection at the right moment.

> **Sound Stamp hard privacy rules: audio buffer never written to disk. Max in-memory lifetime ~500ms. Never transmitted under Mode 0, 1, or 2. Buffer discarded before any AI mode decision is evaluated. iOS orange mic indicator will appear in Still mode when Sound Stamp is enabled — this is intentional and must not be suppressed.**

11. Non-Functional Requirements

11.1 Performance Targets

Requirement	Target	Min device	Measurement
Cold app launch → capture-ready	< 1.5 seconds	iPhone 15	XCTest application launch metric
Capture → preview latency (Still)	< 300ms	iPhone 15	CACurrentMediaTime() diff: shutter → preview render
Mode switch animation	< 150ms	iPhone 15	Gesture recognition → ghost label first frame
Preset cycle (swipe on bar)	< 120ms crossfade	iPhone 15	Gesture recognition → full accent color update across all 4 surfaces
Journal load (5,000 assets)	< 2 seconds	iPhone 15	Time to first Moment card rendered
On-device indexing (per asset)	< 3 seconds (background, charging)	iPhone 15	Capture timestamp → vibe tags written to graph
Sound Stamp analysis	< 300ms from buffer end	iPhone 15	Timestamp diff: buffer end → GraphManager write complete
Fix modal open	< 200ms from tap	iPhone 15	Tap event → modal fully presented with photo loaded
Fix derivative write	< 1 second from Done tap	iPhone 15	Done tap → Fixed badge visible on thumbnail
Reel assembly (15 assets)	< 20 seconds on-device	iPhone 15	User trigger → preview-ready Reel
App memory (active capture)	< 200 MB	iPhone 15	Instruments Memory profiler during Clip recording
Graph export to App Group	< 2 seconds	iPhone 15	graphManager.exportForCompanion() duration

11.2 Code Quality Requirements

- Test coverage: NiftyCore domain layer ≥ 85% line coverage. Engine logic ≥ 80%. Platform adapters in NiftyData ≥ 60% (hardware-dependent code excluded).

- SwiftLint: all targets must pass with project .swiftlint.yml. Zero errors in CI. Warnings addressed within same sprint.
- No direct framework imports above data layer: CI check validates zero UIKit/SwiftUI/AVFoundation/CoreML imports in NiftyCore/Sources/. Violation blocks merge.
- Protocol-first: any new cross-layer capability must be introduced as a protocol in Domain/Protocols/ before any implementation is written.
- Actor-isolated mutations: any mutable shared state must be isolated to an actor. No unstructured Tasks that write to shared state.

11.3 Supported Devices & OS

> **CORRECTED from prior SRS draft: minimum platform is iOS 26.0 / iPhone 15 — NOT iOS 17.2 / iPhone 13. Required for updated AI Intelligence frameworks, updated AVFoundation APIs, Neural Engine performance for updated on-device ML, and computational photography APIs.**

Requirement	Specification	Notes
Minimum iOS	iOS 26.0	Required for updated AI Intelligence frameworks, updated AVFoundation AVCaptureMultiCamSession APIs, and computational photography APIs.
Minimum device	iPhone 15	Neural Engine performance for updated on-device ML. iPhone 13 minimum from PRD v1.4 is no longer supported.
Target devices	iPhone 15 and 16 series	Primary development and QA target.
Swift version	Swift 5.9+	Required for macro support and improved actor isolation.
Xcode version	Xcode 16+	Required for iOS 26 SDK.
iPadOS	Not supported v1.0	Evaluate at v2.0 milestone.
macOS Catalyst	Not in scope	Not in scope v1.0.

12. Still Photo Fix — Feature Specification

> **Scope principle: Fix is a correction tool, not a creative editor. Limited to geometry only (crop, rotate, flip). Never exposes brightness, contrast, saturation, healing, or any other adjustment. The word 'Edit' must never appear anywhere in this flow — all UI uses 'Fix' (verb) and 'Fixed' (past state). AppConfig gate: config.features.contains(.photoFix).**

12.1 Asset Eligibility

Asset type	Fix available?	Behaviour
Still (HEIF/JPEG)	Yes — crop, rotate, flip	Full Fix. Derivative written to vault.
Live (HEIF + MOV)	Yes — Still frame only	Fix applies to HEIF component. MOV motion component is not affected. Disclosure shown: 'Motion clip not affected.'
Clip, Echo, Atmosphere	No	Fix option does not appear in context menus or viewers for these types.

12.2 Entry Points

- Long-press asset in Moment detail grid → context menu: Share · Set as hero · Fix · Delete. Fix shown for Still/Live only.
- Full-screen asset viewer → 'Fix' pill button (bottom-right). Still/Live only.
- Both entry points open the Fix modal as a full-screen push from the right.

12.3 Fix Modal

Element	Specification
Transition	Full-screen push from right. Open latency < 200ms.
Top bar	Cancel (left) · Fix (center title — never 'Edit') · Done in purple #5B4FCF (right).
Photo canvas	Full-screen. Active vibe preset LUT applied in real time throughout.
Tool bar	Liquid Glass (UIVisualEffectView .systemUltraThinMaterialDark). Tools: Crop · Rotate 90° · Flip. Active tool: filled pill.
Revert link	Always visible: 'Revert to original'.
Live disclosure	When Live asset: one-line note in tool bar: 'Motion clip not affected.'

12.4 Correction Specifications

Tool	Gesture	Behaviour	Constraints

Crop	Drag edge or corner handle. Pinch to zoom/reposition within frame.	L-handle corners (3pt, rounded). Crop frame updates in real time. Dimmed overlay on excluded area. Rule-of-thirds grid always visible. Double-tap photo resets to full frame.	Min 200×200pt. Free aspect ratio — no locked ratios in v1.0.
Rotate 90°	Single tap on Rotate tool. Repeatable.	90° clockwise per tap. Spring animation 200ms. Crop frame auto-adjusts. Grid reorients. 4 taps = full rotation.	90° increments only. No free-angle dial.
Flip	Tap Flip tool → inline sub-row: Horizontal · Vertical.	Mirror animation 150ms. H and V can both be active. Second tap on active option reverses it. Flip + Rotate compound correctly.	Revert clears all corrections including flip state.

12.5 Derivative Asset Lifecycle

Rule	Specification
Done with changes	Writes derivative to vault alongside original. Original never modified or deleted.
Done with no changes	Identical to Cancel. No derivative written.
Derivative = primary	Derivative is primary displayed version in: journal, Moment detail, widgets, share flows.
Write latency	< 1 second from Done tap to Fixed badge visible.
Graph flags	fix_applied: true, is_derivative: true, source_asset_id in asset_scores table.
Metadata inheritance	Derivative inherits all vibe tags, acoustic tags, ambient metadata. Fix does not re-trigger indexing.
Roll Mode / streak	Derivative does not count as a new asset.
Lab / Enhanced AI	Always operates on derivative if one exists.
iCloud offload	Always-local. Excluded from Smart Archive pipeline. (OI-002 resolved.)
Fixed badge	Small 'Fixed' text badge on thumbnail. Tap shows: 'Original preserved · Revert.'
Share	Derivative shared by default. 'Fixed' label shown in share preview. No share-original option in share flow.
Revert	Confirmation: 'Remove fix? Your original photo will be shown.' Confirming deletes derivative, restores original as primary.

12.6 Acceptance Criteria — Fix

AC #	Criterion	Test type
AC-FIX-01	Fix available on Still and Live assets only. Does not appear for Clip, Echo, Atmosphere.	Manual / UI

AC-FIX-02	Accessible from both entry points: long-press context menu and full-screen pill button.	Manual / UI
AC-FIX-03	Fix modal top bar: Cancel · Fix (never 'Edit') · Done in purple.	Manual / UI
AC-FIX-04	Crop: drag handles, dimmed overlay, rule-of-thirds grid, double-tap reset, 200×200pt min, free aspect ratio.	Manual / UI
AC-FIX-05	Rotate 90°: single tap, 200ms spring animation, crop auto-adjusts, no dial.	Manual / UI
AC-FIX-06	Flip: sub-row H/V, 150ms mirror animation, both compoundable, second tap reverses, compounds with rotate.	Manual / UI
AC-FIX-07	Done with changes: derivative written, original preserved, Fixed badge appears within 1 second.	Manual / XCTest
AC-FIX-08	Done with no changes = Cancel. No derivative written.	Manual / UI
AC-FIX-09	Revert: confirmation shown, derivative deleted, original restored as primary.	Manual / UI
AC-FIX-10	Live asset: HEIF fixed, MOV unaffected, 'Motion clip not affected' disclosed.	Manual / UI
AC-FIX-11	Derivative inherits all tags. No re-indexing. Does not count for Roll Mode/streak.	XCTest / Unit
AC-FIX-12	Modal opens < 200ms. Derivative write + badge < 1 second.	XCTest / Performance

13. Sound Stamp — Feature Specification

> **Privacy hard rule: Sound Stamp audio is never written to disk under any condition or AI mode. The 1.5s buffer exists only in memory, passes through the on-device acoustic classifier, and is released. It is metadata infrastructure — not an asset. Never stored, never exported, never transmitted. AppConfig gate: config.features.contains(.soundStamp).**

13.1 Overview

Sound Stamp is an opt-in feature that captures a 1.5s ambient audio buffer during every Still capture (0.5s pre-roll + 1.0s post-shutter), analyses it on-device using the same Core ML acoustic classifier as Echo/Clip/Atmosphere, writes acoustic vibe tags to the intelligence graph, and immediately discards the buffer. No audio file is ever created.

Sound Stamp closes the acoustic tagging gap for Still-only users — prior to v1.5, Still assets received zero acoustic vibe enrichment.

13.2 Audio Buffer Technical Specification

Parameter	Specification
Buffer duration	1.5s total: 0.5s pre-roll (always-on buffer) + 1.0s post-shutter capture.
Audio format	PCM 44.1kHz in-memory buffer only. Never encoded. Never written to disk. No file container.
Pre-roll model	Always-on when Sound Stamp enabled in Settings AND app foregrounded in Still mode. Same buffering model as Apple Live Photo. (OI-001 resolved — Option A.)
Microphone session	Activates on entry to Still mode. Deactivates on mode switch, app background, or Settings toggle-off.
iOS orange mic indicator	Will appear in Still mode while Sound Stamp is enabled. Intentional and required — system-level privacy signal. Must not be suppressed.
Processing engine	On-device SoundAnalysis.SNClassifySoundRequest (via SoundStampAdapter implementing SoundStampPipelineProtocol). Same Core ML model as §6.2 Step 3.
Analysis latency	< 300ms from buffer end. Background thread. Does not block UI or capture pipeline.
Buffer lifecycle	Allocated at shutter fire → classified → released immediately after tag write. Max in-memory lifetime ~500ms.
No file creation	Under no condition is an audio file of any format created, staged, or written.
No transmission	Buffer discarded before any AI mode decision. Never transmitted under Mode 0, 1, or 2.

13.3 Opt-In Model & Permission

Surface	Element	Default	Content

Onboarding Step 5	Sound Stamp toggle	Off	'Capture 1.5 seconds of ambient sound with every photo — niftyMomnt uses it to tag the vibe (rainy, crowded, quiet) and immediately discards the audio. Nothing is stored or shared.'
Settings → Capture → Sound Stamp	Toggle + sub-label	Off	Sub-label: '1.5s ambient audio at capture. Analysed on-device, never stored.' Learn more: 3-line expansion.
Mic permission denied	Toggle greyed out + note	N/A	'Requires microphone access — enable in iOS Settings.' Tapping navigates to iOS Settings.

13.4 Capture Hub UI States

State	UI element	Specification
Idle — Sound Stamp ON	≋ icon inside shutter button	Still mode only. Active preset accent color at 70% opacity.
Idle — Sound Stamp OFF	No indicator	No change to shutter button.
Post-capture (0–1.5s)	3 concentric arcs pulsing from shutter ring	Preset accent color fading to transparent. Fixed amplitude — not reactive to audio level.
Reduced Motion	Static arc	30% opacity, 1s, then fades. No animation.
Analysis complete	No feedback	Tags written silently to asset metadata.

13.5 Acoustic Tags & Storage

Tag	Detectable in 1.5s?	Confidence threshold	Notes
Windy	Yes	≥ 0.5	Broadband noise — near instant.
Crowded	Yes	≥ 0.5	Overlapping voice density detectable < 1s.
Music	Yes	≥ 0.7	Higher threshold — lower confidence for distant/ambient music.
Quiet / Serene	Yes	≥ 0.5	Low amplitude + minimal transients.
Rain	Yes	≥ 0.5	Rhythmic broadband — detectable < 1s.
Ocean / Water	Partial	≥ 0.5	Low-frequency wave needs full 1.5s. May be enriched by later Echo/Atmosphere in same Moment cluster.

Tag storage: vibe_tags table, source field: 'sound_stamp'. Treated identically to echo/clip/atmosphere acoustic tags by all downstream features (Nudge Engine, Reel assembler, Mood Map, Annual Review).

13.6 Self-Timer & Dual-Camera Interactions

- Self-timer: pre-roll buffer active during countdown. Analysis fires at shutter completion (end of countdown).
- Dual-camera contextual capture: Sound Stamp and secondary camera capture fire at the same shutter event. Processing is parallel — neither blocks the other.

13.7 Acceptance Criteria — Sound Stamp

AC #	Criterion	Test type
AC-SS-01	Off by default. Activatable only via Onboarding Step 5 or Settings toggle.	Manual / UI
AC-SS-02	≋ icon in shutter button in Still mode only when enabled (preset accent color, 70% opacity). No icon in other modes.	Manual / UI
AC-SS-03	1.5s buffer captured (0.5s pre-roll + 1.0s post-shutter). Analysed by same Core ML classifier as Echo/Clip/Atmosphere.	XCTest / Integration
AC-SS-04	Buffer never written to disk, never stored, never exported, never transmitted under any mode. Max in-memory lifetime ~500ms.	XCTest / Security — filesystem + network inspection
AC-SS-05	Post-capture: 3 concentric arcs pulse 1.5s in preset accent color, fixed amplitude. Reduced Motion: static arc 30% opacity 1s then fades.	Manual / UI
AC-SS-06	Acoustic tags written within 300ms of buffer end, without blocking UI. Tags visible in Moment detail vibe panel.	XCTest / Performance
AC-SS-07	Mic permission denied: toggle greyed out with 'Requires microphone access' note. Never silently inactive.	Manual / UI
AC-SS-08	Sound Stamp tags feed Nudge Engine, Reel assembler, Mood Map, Annual Review identically to Echo acoustic tags.	XCTest / Integration

14. Resolved Open Issues

ID	Issue	Decision	Impact on code
OI-001	Sound Stamp pre-roll: always-on when foregrounded in Still mode, or finger-on-shutter only?	RESOLVED — Option A: Always-on. Pre-roll buffer active whenever Sound Stamp is enabled in Settings AND app is foregrounded in Still mode. Microphone session activates on Still mode entry; deactivates on mode switch, app background, or Settings toggle-off. iOS orange mic indicator will appear — intentional, must not be suppressed.	SoundStampAdapter.activatePreRoll() called from CaptureEngine on Still mode entry. deactivatePreRoll() called on mode exit. AVAudioSession category management in SoundStampAdapter. CaptureEngine must call deactivate on scenePhase change to .background.
OI-002	Fix derivatives: included in iCloud Photo offload / Smart Archive, or always-local?	RESOLVED — Option A: Always-local. Derivatives excluded from Smart Archive BGAppRefreshTask and iCloud Photo offload pipeline. Both original and derivative retained on-device at all times. Option C (offload original, keep derivative local) earmarked as v1.5 upgrade path.	VaultRepository Smart Archive BGAppRefreshTask must filter out assets where is_derivative = true. No code path in VaultRepository may include derivative files in archive batch.